

AI-Supported Dark Web Data Breach Monitoring Automation

Berke Taha Özdemir

ORCID: 0009-0008-3670-0152

244410043

Department of Computer Engineering

Kastamonu University

Kastamonu, Türkiye

berketahaoz@gmail.com

Abstract—This report explains the architecture, implementation steps, and results of an AI-supported dark web data breach monitoring automation developed for the Database Management Systems course. Without connecting to the real dark web, the system processes educational mock breach records through n8n; uses Google Gemini to generate risk level, category, summary, and critical-domain information; and stores the results in normalized Microsoft SQL Server tables. On the database side, idempotent insertion is supported by a stored procedure, critical-domain alerts are produced by a trigger, and search performance is improved with indexes. A Node.js/Express dashboard presents the database records visually. The project combines cybersecurity awareness and relational database design within a safe, controlled student project.

Keywords—n8n, Google Gemini, SQL Server, SSMS, data breach, AI automation, dark web monitoring, trigger, stored procedure, 3NF, Node.js dashboard

I. INTRODUCTION

Data breaches are no longer a problem only for large companies; they also affect universities, public institutions, and smaller organizations. When an email address, username, or password hash is shared in underground environments, it may later be used for credential stuffing, social engineering, targeted phishing, or account takeover attempts. Therefore, breach records should not only be stored, but also interpreted, categorized by risk, and used to generate alerts when needed [3]-[7].

This study was designed and implemented as a dark web data breach monitoring simulation. The goal is not to collect real dark web data. Instead, the goal is to automatically process sample records assumed to come from dark web sources, classify them with Google Gemini, and store them in a relational SQL Server design. In this way, automation, AI-supported analysis, and database management topics such as normalization, triggers, procedures, unique constraints, and indexes are combined in a single scenario [11]-[18].

II. CONTRIBUTION

The main contribution of this project is that it translates the idea of data breach monitoring into a safe educational environment without connecting to the real dark web or collecting real personal data.

The second contribution is the integration of multiple layers in an end-to-end workflow: n8n processes mock breach records, Gemini classifies the description, SQL Server stores the data according to a 3NF-oriented design, stored procedures and triggers enforce business rules, and a Node.js/Express dashboard presents the output.

The third contribution is the explicit preservation of secure design boundaries. The project works only with simulated records, does not store plaintext passwords, and focuses on defensive scenarios such as critical-domain alerting. In this respect, it is compatible with secure-by-design principles and NIST incident response logic [9], [10], [19], [20].

III. LITERATURE REVIEW

Recent studies show that dark web monitoring is not merely a single search query. Forums, paste sites, marketplaces, and breach announcement channels produce different data types. Dark Watchdog proposes a RAG-driven approach for near-real-time detection and analysis of data-leak posts on dark web forums [1]. DarkBERT shows that dark web language differs from surface-web language and that domain-specific models can be useful for cybersecurity tasks [2].

On the organizational side, dark web monitoring is discussed as an early-warning and threat-intelligence strategy [3]. Studies on leaked credentials focus on password reuse, compromised credential checking, and post-leak attack behavior. A 2025 honeypot study is important because it shows that stolen credentials may continue to be used in attack chains after the original leak [4]. Privacy-preserving compromised credential checking protocols aim to protect query privacy while checking whether an account appears in a compromised credential database [5].

Current industry reports also support the importance of data breaches and identity security. Verizon's 2026 DBIR emphasizes AI-driven speed and vulnerability exploitation, while IBM's 2025 report highlights the financial impact of data breaches and the risks of weak AI governance [6], [7]. ENISA's 2025 threat landscape report also shows that monitoring capabilities must be kept current [8].

IV. PROBLEM DEFINITION

A breach record usually consists of a source name, title, description, date, affected users, email addresses, hash values, and risk information. Keeping all of these fields in a single table may appear simple at first, but repeated source names, repeated breach titles, and repeated user information can create redundancy and inconsistency.

In this project, the problem is limited to interpreting the n8n-generated record with AI, storing it without unnecessary duplication in SQL Server, and automatically marking critical domains as alerts. Since the system does not collect real dark web data, the legally risky data collection stage is excluded from the scope.

V. METHOD AND TECHNOLOGIES

The project uses n8n, Google Gemini, Microsoft SQL Server, SSMS, and a Node.js/Express dashboard. On the n8n side, the workflow contains a manual trigger, mock data generation, AI analysis, parsing, and a Microsoft SQL Server insertion step. n8n's Microsoft SQL integration supports operations such as executing SQL queries and inserting rows, making it suitable as the bridge between automation and the database [12].

The Google Gemini layer generates the risk level, category, short summary, and critical-domain result from the breach description. Gemini's structured-output approach is suitable for extracting specific fields from text and binding classification results to a more predictable schema [11].

SQL Server and SSMS are used for storing, querying, and demonstrating the results. The stored procedure supports controlled

insertion, the trigger supports critical-domain alerting, and indexes support search performance [13]-[15]. The dashboard layer connects to the database in read-only mode through the mssql package.

VI. SYSTEM ARCHITECTURE

The system is centered on an n8n workflow. The workflow first generates a sample breach record. Google Gemini analyzes the record. The parse step converts the AI response into fields expected by the SQL Server stored procedure. The Microsoft SQL node calls `sp_InsertBreachRecord` and inserts the data into the database. Critical-domain detection is handled by a database trigger rather than the application layer. The dashboard connects to the database only for reading and visualization.

```
Manual Trigger -> Generate Mock Breach Data ->
Google Gemini AI Analysis -> Parse Gemini Output ->
SQL Stored Procedure -> SQL Trigger -> Node.js
Dashboard
```

This separation is important. Even if n8n sends the same data again, the unique constraints and stored procedure checks on the database side prevent unnecessary duplicates.

VII. DATABASE DESIGN

The database design aims to store each piece of information in the table where it naturally belongs. Source information is stored in `Sources`, breach events in `Breaches`, leaked users in `Leaked_Users`, AI output in `AI_Analysis`, and critical-domain alerts in `Critical_Alert_Logs`. Automation runs are tracked in `Automation_Runs`.

A. Tables and Responsibilities

- `Sources`: Stores mock feeds, RSS sources, or API sources.
- `Breaches`: Stores breach title, date, description, and source reference.
- `Leaked_Users`: Stores email addresses and password hashes; plaintext passwords are not stored [19].
- `AI_Analysis`: Stores Gemini AI output such as risk level, category, summary, and critical-domain information.
- `Critical_Alert_Logs`: Stores alerts generated by the trigger when a critical domain is detected.
- `Automation_Runs`: Stores workflow/procedure execution status, processed count, and error messages.

B. 3NF Approach

Repeated fields are separated in the design. Source name and URL are not repeated inside `Breaches`; only `source_id` is stored as a foreign key. Similarly, `Leaked_Users` does not store breach titles or source URLs. AI output is not embedded in the user table but stored separately in `AI_Analysis`. Microsoft's normalization explanation also describes the first three normal forms as a practical level for most applications [16].

VIII. AUTOMATION AND AI ANALYSIS LAYER

The n8n workflow is configured to run manually during the demo. The Gemini AI Analysis node receives the title, description, email, and hash from the mock breach record and produces a JSON-like result containing risk level, category, short summary, and critical-domain information. The parse step converts the AI response into SQL Server fields.

The AI layer is not the only decision-making mechanism. It mainly converts the breach description into a readable classification. Final persistence and business rules are enforced by SQL Server

constraints, stored procedures, and triggers. This prevents data integrity from depending solely on LLM output.

IX. DATABASE MECHANISMS

A. Stored Procedure

`sp_InsertBreachRecord` is used to insert n8n data in a controlled way instead of directly executing `INSERT` statements. The procedure first checks whether the source exists, then checks the breach record, and finally prevents inserting the same email under the same breach more than once. This provides idempotent behavior [13].

B. Trigger

The trigger runs after a new record is inserted into `Leaked_Users`. If the email address contains `.edu`, `.edu.tr`, `.gov`, or `.gov.tr`, an alert is inserted into `Critical_Alert_Logs`. Keeping this rule inside the database ensures that critical-domain detection remains active even if the n8n workflow changes [14].

C. Index and Constraint

Because email-based searches are common, an index is used on `Leaked_Users.email`. An index on `breach_id` also helps list all users associated with a breach. Primary-key and foreign-key relationships provide referential integrity, while unique constraints prevent duplicate records [15], [17], [18].

X. DASHBOARD AND APPLICATION RESULTS

A local Node.js/Express dashboard was prepared for the project. The dashboard shows SQL Server data as statistic cards and tables. It includes TR/EN language selection, refresh button, email search, and risk-level filter. The dashboard does not create data; it only reads the `DarkWebBreachMonitoringDB` database.

The demo result showed `Sources=1`, `Breaches=5`, `Leaked Users=5`, `AI Analyses=5`, `Critical Alerts=4`, and `Automation Runs=11`. The record `gemini.researcher@university.edu.tr` appeared in `Leaked_Users`. Gemini analysis saved the record with `HIGH` risk and `Education` category, and the `.edu.tr` domain automatically generated a critical alert.

XI. SECURITY AND ETHICAL APPROACH

This project did not connect to the real dark web and did not collect real leaked data. The data used in the system is simulated for educational purposes. This decision is important both legally and ethically. The system also does not store plaintext passwords; it only uses hash-like values. NIST SP 800-63B-4 defines current authentication and password-management expectations, and avoiding plaintext password storage is a basic security requirement [19].

The project demonstrates detection, recording, and alerting functions in a small scale, similar to the Detect and Respond logic in the NIST Cybersecurity Framework [9], [10]. CISA's secure-by-design approach also emphasizes that software security should be a core part of design rather than a later add-on [20].

XII. CONCLUSION

This study implements the idea of dark web data breach monitoring as a safe automation project without collecting real data or entering risky environments. The system combines n8n automation, Gemini text/risk analysis, SQL Server relational data management, and Node.js dashboard visualization.

The design directly uses database topics such as 3NF, foreign keys, unique constraints, stored procedures, triggers, and indexes. Therefore, the project is a working application that makes the data breach monitoring scenario concrete from both cybersecurity and database architecture perspectives.

XIII. FUTURE WORK

Future work may define an explicit JSON schema on the Gemini side to make AI output more stable. This would make risk level, category, affected domain, and short summary fields more predictable [11]. The number of mock data sources can also be increased to test different breach scenarios, industries, and risk levels.

The next version may add role-based dashboard access, audit logs, detailed filtering, timeline charts, and critical-alert history. On the database side, more detailed index strategies, archive tables, and backup scenarios may be tested. At institutional scale, the system could be extended with SIEM/SOAR integration, secure API sources, and lawful threat-intelligence feeds. Such extensions should be limited to authorized, defensive, and ethical data sources.

REFERENCES

- [1] S.-L. Hung, C.-K. Chen, K. Furumoto, T. Takahashi and H.-M. Sun, "Dark Watchdog: A Novel RAG-Driven System for Real-Time Detection and Analysis of Data Leaks on Dark Web Forums," 2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops), pp. 11-19, 2024. DOI: 10.1109/ACSACW65225.2024.00010. Publication date: 2024.
- [2] Y. Jin, E. Jang, J. Cui, J.-W. Chung, Y. Lee and S. Shin, "DarkBERT: A Language Model for the Dark Side of the Internet," Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 7515-7533, 2023. DOI: 10.18653/v1/2023.acl-long.415. Publication date: 2023.
- [3] A. Aslan, "Dark Web Monitoring as an Emerging Cybersecurity Strategy for Businesses," International Journal of Information Engineering and Electronic Business, vol. 16, no. 2, pp. 52-65, 2024. Publication date: 2024.
- [4] M. Rabzelj and U. Sedlar, "Beyond the Leak: Analyzing the Real-World Exploitation of Stolen Credentials Using Honey pots," Sensors, vol. 25, no. 12, 3676, 2025. DOI: 10.3390/s25123676. Publication date: June 12, 2025.
- [5] X. Yu, D. Tang, Z. Zhao and W. Zhao, "Privacy-preserving compromised credential checking protocol for account protection," Computer Standards & Interfaces, vol. 89, article 103823, 2024. DOI: 10.1016/j.csi.2023.103823. Publication date: April 1, 2024.
- [6] Verizon, "2026 Data Breach Investigations Report," Verizon Business, 2026. Publication date: May 19, 2026. [Online]. Available: <https://www.verizon.com/business/resources/reports/dbir/>. Accessed: May 25, 2026.
- [7] IBM Security and Ponemon Institute, "Cost of a Data Breach Report 2025," IBM, 2025. Publication date: 2025. [Online]. Available: <https://www.ibm.com/reports/data-breach>. Accessed: May 25, 2026.
- [8] ENISA, "ENISA Threat Landscape 2025," European Union Agency for Cybersecurity, 2025. Publication date: October 1, 2025. [Online]. Available: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025>. Accessed: May 25, 2026.
- [9] National Institute of Standards and Technology, "The NIST Cybersecurity Framework (CSF) 2.0," NIST CSWP 29, 2024. Publication date: February 26, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/cswp/29/the-nist-cybersecurity-framework-csf-20/final>. Accessed: May 25, 2026.
- [10] A. Nelson, S. Rekhi, M. Souppaya and K. Scarfone, "Incident Response Recommendations and Considerations for Cybersecurity Risk Management," NIST SP 800-61 Rev. 3, 2025. Publication date: April 2025. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/61/r3/final>. Accessed: May 25, 2026.
- [11] Google Developers Blog, "Improving Structured Outputs in the Gemini API," Google, 2025. Publication date: November 5, 2025. [Online]. Available: <https://blog.google/innovation-and-ai/technology/developers-tools/gemini-api-structured-outputs/>. Accessed: May 25, 2026.
- [12] n8n, "Create workflows with Microsoft SQL integrations," n8n, 2025. Publication date: August 28, 2025. [Online]. Available: <https://n8n.io/integrations/microsoft-sql/>. Accessed: May 25, 2026.
- [13] Microsoft, "CREATE PROCEDURE (Transact-SQL)," Microsoft Learn, 2025. Last updated: November 18, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-procedure-transact-sql>. Accessed: May 25, 2026.
- [14] Microsoft, "CREATE TRIGGER (Transact-SQL)," Microsoft Learn, 2025. Last updated: November 18, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/sql/t-sql/statements/create-trigger-transact-sql>. Accessed: May 25, 2026.
- [15] Microsoft, "Indexes - SQL Server," Microsoft Learn, 2025. Last updated: November 18, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/sql/relational-databases/indexes/indexes>. Accessed: May 25, 2026.
- [16] Microsoft, "Database normalization description," Microsoft Learn, 2025. Last updated: June 25, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/troubleshoot/microsoft-365-apps/access/database-normalization-description>. Accessed: May 25, 2026.
- [17] Microsoft, "Primary and foreign key constraints - SQL Server," Microsoft Learn, 2025. Last updated: November 18, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints>. Accessed: May 25, 2026.
- [18] Microsoft, "Unique constraints and check constraints - SQL Server," Microsoft Learn, 2025. Last updated: November 18, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/sql/relational-databases/tables/unique-constraints-and-check-constraints>. Accessed: May 25, 2026.
- [19] D. Temoshok, E. Chuvakin, J. Fenton, N. Regenscheid, S. Stine, J. Richer and R. Choong, "Digital Identity Guidelines: Authentication and Authenticator Management," NIST SP 800-63B-4, 2025. Publication date: July 2025. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/63/b/4/final>. Accessed: May 25, 2026.
- [20] CISA, NSA and international partners, "Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Secure by Design Software," CISA, 2023. Updated: October 2023. [Online]. Available: <https://www.cisa.gov/securebydesign>. Accessed: May 25, 2026.